

Embedded Translation Interface: Arduino Hardware Prototype

Physicalizing the Korean-Nepali Swap Dictionary

Danish Puri

Instructor

June 18, 2026

A Message from Danish to the Hardware Team

Welcome to the physical realm! Writing software on a PC gives you infinite memory. Moving your translation engine to an **Arduino Uno** gives you exactly 2 Kilobytes of RAM. To solve this, we are building a physical macro-pad. You will wire buttons to represent Korean concepts. When pressed, the Arduino handles the physical logic, flashes status LEDs, and bridges the hardware to a PC Serial Monitor to render the native Devanagari translation fonts. Let's start building.

Contents

1	Bill of Materials (Hardware Required)	2
1.1	Pro-Tip: Internal Pull-Up Resistors	2
2	Arduino Embedded C++ Firmware	2
3	Next Steps for the Engineering Team	4

1 Bill of Materials (Hardware Required)

To construct the Liberty College baseline prototype, sign out the following components from the hardware lab:

- **1x** Arduino Uno R3 (or compatible clone)
- **1x** Half-size or Full-size Solderless Breadboard
- **3x** Tactile Push Buttons (For our 3 core words: *I, Water, Drink*)
- **1x** 5mm LED (Any color—acts as the "Translation Successful" indicator)
- **1x** 220Ω Resistor (To protect the LED)
- **Jumper Wires** (Male-to-Male)

1.1 Pro-Tip: Internal Pull-Up Resistors

Normally, buttons require bulky 10kΩ external resistors to prevent "floating" pins. We are going to write smarter code to use the Arduino's hidden, built-in resistors (INPUT_PULLUP), keeping our breadboard remarkably clean.

2 Arduino Embedded C++ Firmware

Upload this C++ sketch to your Arduino using the IDE. Open the **Serial Monitor** and set the baud rate to 9600. Because the IDE Serial Monitor is run by Java, it natively supports rendering Unicode Hangul and Devanagari text streams!

```
1  /*
2   Liberty College - Hardware Translation Interface
3   Author: Danish Puri
4   Hardware: 3 Buttons on Pins 2,3,4. LED on Pin 13.
5  */
6
7  // Hardware Pin Definitions
8  const int buttonNa = 2;    // Button representing "" (I)
9  const int buttonMul = 3;   // Button representing "" (Water)
10 const int buttonMas = 4;   // Button representing "" (Drink)
11 const int statusLED = 13;  // Built-in LED / Breadboard LED
12
13 // Debounce Variables to prevent double-clicking
14 unsigned long lastDebounceTime = 0;
15 const unsigned long debounceDelay = 200;
16
17 void setup() {
18   // Initialize PC Communication at 9600 bits per second
19   Serial.begin(9600);
20
21   // Set up the LED as an Output
22   pinMode(statusLED, OUTPUT);
```

```
23 digitalWrite(statusLED, LOW);
24
25 // Set up Buttons using INTERNAL PULLUP resistors
26 // Note: Buttons will read LOW (0) when pressed, HIGH (1) when unpressed!
27 pinMode(buttonNa, INPUT_PULLUP);
28 pinMode(buttonMul, INPUT_PULLUP);
29 pinMode(buttonMas, INPUT_PULLUP);
30
31 Serial.println("=====");
32 Serial.println("Liberty College Translation Hardware Online.");
33 Serial.println("Awaiting Physical Input...");
34 Serial.println("=====\\n");
35 }
36
37 void processTranslation(String koreanWord, String nepaliWord) {
38     // 1. Flash the LED to show hardware processed the interrupt
39     digitalWrite(statusLED, HIGH);
40
41     // 2. Output the Unicode translation to the Serial Monitor
42     Serial.print("INPUT DETECTED [KR]: ");
43     Serial.print(koreanWord);
44     Serial.print(" --> [NP]: ");
45     Serial.println(nepaliWord);
46
47     // 3. Keep LED on briefly, then turn off
48     delay(300);
49     digitalWrite(statusLED, LOW);
50 }
51
52 void loop() {
53     // Read current hardware state
54     int stateNa = digitalRead(buttonNa);
55     int stateMul = digitalRead(buttonMul);
56     int stateMas = digitalRead(buttonMas);
57
58     // Check if a button is pressed (LOW) and delay has passed
59     if (millis() - lastDebounceTime > debounceDelay) {
60
61         if (stateNa == LOW) {
62             processTranslation(" (Na)", " (Ma)");
63             lastDebounceTime = millis();
64         }
65
66         else if (stateMul == LOW) {
67             processTranslation(" (Mul)", " (Paani)");
68             lastDebounceTime = millis();
69         }
70
71         else if (stateMas == LOW) {
72             processTranslation(" (Masinda)", " (Piunchhu)");
```

```
73     lastDebounceTime = millis();  
74   }  
75 }  
76 }
```

Listing 1: Arduino Firmware (main.ino)

3 Next Steps for the Engineering Team

Once this basic interface works, here are the modules you can research to expand the system from a prototype into a standalone device:

1. **SD Card Module Adapter (SPI):** To bypass the Arduino's 2KB memory limit, we can store our `dictionary.json` file on a MicroSD card. The Arduino will read the file line-by-line to search for translations without crashing its RAM.
2. **Matrix Keypad:** Replace the 3 buttons with a 4x4 membrane keypad (16 buttons) to build a larger hardware dictionary matrix.
3. **TFT SPI Display:** Swap the PC Serial Monitor out for a 2.4" TFT Display. Note: You will need to flash a custom Unicode font library (like U8g2) to successfully draw Hangul and Devanagari pixels on the screen.